Sugar Trap: Finding the Blue Ocean in Healthy Snacking

Objective:

Identify underserved healthy snack opportunities using Open Food Facts data

```
!wget -O openfoodfacts.csv.gz https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz
```

```
--2026-06-22 01:28:34--  https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz
Resolving static.openfoodfacts.org (static.openfoodfacts.org)... 151.115.132.10
Connecting to static.openfoodfacts.org (static.openfoodfacts.org)|151.115.132.10|:443... connected.
HTTP request sent, awaiting response... 302 Moved Temporarily
Location: https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.products.csv.gz [following]
--2026-06-22 01:28:35--  https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.products.csv.gz
Resolving openfoodfacts-ds.s3.eu-west-3.amazonaws.com (openfoodfacts-ds.s3.eu-west-3.amazonaws.com)... 3.5.205.248, 52.
Connecting to openfoodfacts-ds.s3.eu-west-3.amazonaws.com (openfoodfacts-ds.s3.eu-west-3.amazonaws.com)|3.5.205.248|:44
HTTP request sent, awaiting response... 200 OK
Length: 1275171186 (1.2G) [application/gzip]
Saving to: ‘openfoodfacts.csv.gz’

openfoodfacts.csv.g 100%[===================>]   1.19G  24.3MB/s    in 51s

2026-06-22 01:29:27 (23.8 MB/s) - ‘openfoodfacts.csv.gz’ saved [1275171186/1275171186]
```

```
# importing pandas library for data processing
import pandas as pd

sample = pd.read_csv(
    "openfoodfacts.csv.gz",
    sep="\t",
    compression="gzip",
    nrows=5,
    low_memory=False
)

# checking the available columns
print(sample.columns.tolist())
```

```
['code', 'url', 'creator', 'created_t', 'created_datetime', 'last_modified_t', 'last_modified_datetime', 'last_modified
```

```
cols = [
    "product_name",
    "categories_tags",
    "ingredients_text",
    "sugars_100g",
    "proteins_100g",
    "fiber_100g",
    "fat_100g",
    "nutriscore_grade",
    "nova_group"
]
```

```
df = pd.read_csv(
    "openfoodfacts.csv.gz",
    sep="\t",
    compression="gzip",
    usecols=cols,
    nrows=500000,
    low_memory=False
)
```

```
print(df.shape)
```

```
(500000, 9)
```

```
# viewing the first five rows of the dataset
df.head()
```

	product_name	categories_tags	ingredients_text	nutriscore_grade	nova_group	fat_100g	sugars_100g	fiber_100g	p
0	Limonade artisanale a la rose	NaN	NaN	unknown	NaN	NaN			
1	M&M white	NaN	Weizenmehl, Rapsöl, Speisesalz, 1,7% Meersalz,...	unknown	NaN	NaN			
2	Chocolate n3	NaN	NaN	unknown	NaN	NaN			
3	Pâte de fruits	NaN	NaN	unknown	NaN	NaN	NaN	NaN	
4	Paleta gran reserva - Sierra nevada-	en:beverages-and-beverages-preparations,en:bev...	Thiamin, Biotin, Chromium, Garcinia cambogia f...	unknown	4.0	NaN	NaN	NaN	

Enable browser notifications in Settings to get alerts when executions complete

```
# checking brief info about the data
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500000 entries, 0 to 499999
Data columns (total 9 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   product_name      484211 non-null  object
 1   categories_tags   268078 non-null  object
 2   ingredients_text  266958 non-null  object
 3   nutriscore_grade  498349 non-null  object
 4   nova_group        252080 non-null  float64
 5   fat_100g          115076 non-null  float64
 6   sugars_100g       111635 non-null  float64
 7   fiber_100g        83220 non-null   float64
 8   proteins_100g     115225 non-null  float64
dtypes: float64(5), object(4)
memory usage: 34.3+ MB
```

```
# brief description of the dataset
df.describe()
```

	nova_group	fat_100g	sugars_100g	fiber_100g	proteins_100g
count	252080.000000	1.150760e+05	1.116350e+05	8.322000e+04	1.152250e+05
mean	3.490301	1.647922e+02	8.959110e+04	1.235965e+02	8.723586e+03
std	0.954684	5.174682e+04	2.992952e+07	3.466466e+04	2.945984e+06
min	1.000000	0.000000e+00	0.000000e+00	-1.546239e-01	0.000000e+00
25%	3.000000	6.583761e-01	1.000000e+00	0.000000e+00	2.000000e+00
50%	4.000000	5.000000e+00	4.583333e+00	1.639344e+00	6.060000e+00
75%	4.000000	1.785710e+01	1.600000e+01	3.600000e+00	1.084337e+01
max	4.000000	1.755400e+07	1.000000e+10	1.000000e+07	1.000000e+09

```
# checking missing values
df.isnull().sum().sort_values(ascending=False)
```

	0
fiber_100g	416780
sugars_100g	388365
fat_100g	384924
proteins_100g	384775
nova_group	247920
ingredients_text	233042
categories_tags	231922
product_name	15789
nutriscore_grade	1651

dtype: int64

```
# cleaning the data by dropping missing values
clean_df = df.copy()
```

```
clean_df = clean_df.dropna(
    subset = [
        "product_name",
        "categories_tags",
        "sugars_100g",
        "proteins_100g"
    ])
```



```
# Checking size before and after dropping missing values
print("Before:", len(df))
print("After:", len(clean_df))
```

```
Before: 500000
After: 52857
```

```
# Removing impossible nutrition (outlier filtering) values since values are between 0 and 100 (0-100)
clean_df = clean_df[
    (clean_df["sugars_100g"] >= 0) &
    (clean_df["sugars_100g"] <= 100) &
    (clean_df["proteins_100g"] >= 0) &
    (clean_df["proteins_100g"] <= 100)

]
```

```
# brief description on sugars_100g and proteins_100g columns
clean_df[["sugars_100g", "proteins_100g"]].describe()
```

	sugars_100g	proteins_100g
count	52746.000000	52746.000000
mean	12.233006	7.987432
std	17.814261	8.846205
min	0.000000	0.000000
25%	1.176471	2.200000
50%	4.800000	6.572770
75%	14.000000	10.344828
max	100.000000	100.000000

```
# save the cleaned dataset
clean_df.to_csv("clean_openfoodfacts.csv", index = False)
```

```
# checking the size of the dataset
clean_df.shape
```

```
(52746, 9)
```

```
# checking missing values in each column
clean_df.isnull().sum()
```

	0
product_name	0
categories_tags	0
ingredients_text	12034
nutriscore_grade	0
nova_group	13301
fat_100g	69
sugars_100g	0
fiber_100g	9488
proteins_100g	0

dtype: int64

Story 2: Category Wrangler

```
# viewing the columns
clean_df.columns
```

```
Index(['product_name', 'categories_tags', 'ingredients_text',
       'nutriscore_grade', 'nova_group', 'fat_100g', 'sugars_100g',
       'fiber_100g', 'proteins_100g'],
      dtype='object')
```



```
# checking unique values in categories_tags column
clean_df["categories_tags"].unique()
```

```
array(['en:asian-style-ready-meal',
       'en:beverages-and-beverages-preparations,en:beverages',
       'en:plant-based-foods-and-beverages,en:plant-based-foods,en:condiments,en:spices,en:curry-
powder,en:powder,en:sauce-powder',
       ...,
       'en:condiments,en:sauces,en:meal-sauces,en:pasta-sauces,en:tomato-sauces,en:vegetable-sauces,en:tomato-sauces-
with-vegetables',
       'en:breakfasts,en:spreads,en:sweet-spreads,en:bee-products,en:farming-
products,en:sweeteners,en:honeys,en:lavender-honeys',
       'en:meats-and-their-products,en:meats,en:chicken-and-its-products,en:poultries,en:chickens,en:bbq-
chicken,en:sliced-bbq-chicken'],
      dtype=object)
```

```
# Examining the most common value in categories_tags column
from collections import Counter

all_tags = []

for row in clean_df["categories_tags"].dropna():
    all_tags.extend(str(row).split(","))

tag_counts = Counter(all_tags)

pd.DataFrame(
    tag_counts.most_common(50),
    columns=["tag", "count"]
)
```

Enable browser notifications in Settings to get alerts when executions complete

	tag	count
11	en:dairy-desserts	4106
21	en:breakfast-cereals	2481
22	en:meats-and-their-products	2358
23	en:confectioneries	2221
24	en:salty-snacks	2183
25	en:frozen-foods	2156
26	en:appetizers	1993
27	en:spreads	1918
28	en:groceries	1864
29	en:cheeses	1799
31	en:undefined	1643
38	en:cocoas-and-its-products	1327
39	en:chips-and-fries	1321

```
# Products that contain snack-related tags
snack_keywords = [
    "snacks",
    "sweet-snacks",
    "salty-snacks",
    "biscuits",
    "biscuits-and-cakes",
    "biscuits-and-crackers",
    "confectioneries",
    "candies",
    "chips-and-fries",
    "crisps",
    "nuts-and-their-products",
    "seeds",
    "breakfast-cereals",
    "bars"
]
```

```
# use the snack-related tags to create a dataframe
snack_df = clean_df[
    clean_df["categories_tags"].str.contains("|".join(snack_keywords), case=False, na=False)
].copy()
```

```
# the counts of each keyword in snack_keyword
for keyword in snack_keywords:
    count = clean_df["categories_tags"].str.contains(
        keyword,
        case=False,
        na=False
    ).sum()

    print(keyword, count)
```

```
snacks 9301
sweet-snacks 6038
salty-snacks 2185
biscuits 3064
biscuits-and-cakes 2613
biscuits-and-crackers 1108
confectioneries 2222
candies 1604
chips-and-fries 1321
crisps 1255
nuts-and-their-products 1271
seeds 1160
breakfast-cereals 2481
bars 823
```

```
print(snack_df.shape)
```

```
(13987, 9)
```

```
"""# checking values in categories_tags that contain bar
bar_counts = clean_df["categories_tags"].str.contains(
    "bar",
    case=False,
    na=False
).sum()

print("bar_counts: ", bar_counts)"""
```

```
'# checking values in categories_tags that contain bar\nbar_counts = clean_df["categories_tags"].str.contains(\n    "bar",\n    case=False,\n    na=False\n).sum()\n\nprint("bar_counts: ", bar_counts)'
```

```
# created mapping dictionary
category_mapping = {
    "Snack Bars": [
        "bar"
    ],

    "Sweet Snacks": [
        "sweet-snacks",
        "confectioneries",
        "candies"
    ],

    "Biscuits & Cookies": [
        "biscuits",
```

```
        "biscuits-and-cakes",
        "biscuits-and-crackers"
    ],

    "Salty Snacks": [
        "salty-snacks",
        "crisps",
        "chips-and-fries",
        "salted-snacks",
        "popcorn"
    ],

    "Nuts & Seeds": [
        "nuts-and-their-products",
        "seeds"
    ],

    "Breakfast & Cereal Snacks": [
        "breakfast-cereals"
    ]
}
```



```
# build the assignment function
def assign_category(tags):

    tags = str(tags).lower()

    for category, keywords in category_mapping.items():

        if any(keyword in tags for keyword in keywords):
            return category

    return "General / Unspecified Snacks"
```

```
# applying the function
snack_df["primary_category"] = (
    snack_df["categories_tags"]
    .apply(assign_category)
)
```

```
# checking the number of times each value appears in the dataset
snack_df["primary_category"].value_counts()
```

primary_category	count
Sweet Snacks	5174
Breakfast & Cereal Snacks	2375
Nuts & Seeds	2241
Salty Snacks	1951
Snack Bars	970
General / Unspecified Snacks	828
Biscuits & Cookies	448

dtype: int64

```
# the percentage of the value counts
(
    snack_df["primary_category"]
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)
```

primary_category	proportion
Sweet Snacks	36.99
Breakfast & Cereal Snacks	16.98
Nuts & Seeds	16.02
Salty Snacks	13.95

Enable browser notifications in Settings to get alerts when executions complete

Story 3: Nutrient Matrix

```
analysis_df = snack_df.copy()
```

```
# checking average sugars and proteins in each category created
analysis_df.groupby("primary_category")[["sugars_100g", "proteins_100g"]].mean().round(2)
```

dtype: float64

primary_category	sugars_100g	proteins_100g
Biscuits & Cookies	5.22	8.64
Breakfast & Cereal Snacks	21.19	9.13
General / Unspecified Snacks	17.49	12.18
Nuts & Seeds	5.38	13.85
Salty Snacks	3.63	6.89
Snack Bars	31.00	9.22
Sweet Snacks	37.28	5.08

```
# brief description of sugars and proteins columns in our analysis dataset
analysis_df[
    ["sugars_100g", "proteins_100g"]
].describe()
```

	sugars_100g	proteins_100g
count	13987.000000	13987.000000
mean	22.108174	8.247220
std	21.282509	7.087489
min	0.000000	0.000000
25%	3.448276	4.000000
50%	17.857143	6.666670
75%	35.000000	10.000000
max	100.000000	76.530612

```
# I want to measure how many products exist in the target(healthy) quadrant
analysis_df[(analysis_df["proteins_100g"] >= 10)
      & (analysis_df["sugars_100g"] <= 5)].shape
```

```
(1657, 10)
```

```
# Checking the dominant category
analysis_df[(analysis_df.proteins_100g >= 10)
      & (analysis_df.sugars_100g <= 5)]["primary_category"].value_counts()
```

primary_category	count
Nuts & Seeds	766